Introduction to GitHub and GitHub Codespaces

What is GitHub?

GitHub is a web-based platform where developers store, manage, and collaborate on their code. Think of it like a cloud storage service, but one that is built specifically for software projects. It keeps track of every change you make to your files, so you can always go back to an earlier version if something breaks or if you just want to see what changed over time.

At its core, GitHub uses something called Git, which is a version control system. You do not need to be a Git expert to use GitHub though. For most beginners, knowing just the basics is more than enough to get started.

GitHub is also home to millions of open source projects. People share their code publicly, and anyone can learn from it, use it, or contribute to it. It is one of the most widely used platforms in the software world today.

Key Features of GitHub

- **Repositories:** GitHub stores your entire project (code, files, folders) in what is called a repository, or repo for short. Each repo has its own page, settings, and history.
- **Version History:** Every time you save a change in GitHub, it is recorded as a commit. You can look back at any commit to see exactly what was added, removed, or modified. This makes it really easy to track progress and undo mistakes.
- **Branches:** Branches let you work on new features or experiments without affecting the main codebase. Once you are happy with the changes, you can merge them back into the main branch.

- **Pull Requests:** When you want to propose changes, you create a pull request. Other team members can review the code, leave comments, and approve or request changes before anything gets merged.
- **Issues:** GitHub Issues let you track bugs, feature requests, and tasks. It is a simple but effective way to manage what needs to be done in a project.
- **GitHub Actions:** GitHub Actions lets you automate workflows like testing your code, building your app, or deploying it every time you push a change. It is a built-in CI/CD tool.
- **Access Control:** You can control who has access to your repository and what they can do. Repos can be public (visible to everyone) or private (only visible to people you invite).

> *In this course, we will only use GitHub to host our code and model files. That is it. We are not going to dive deep into pull requests, branching strategies, or any of the more advanced collaboration features. Just the basics to get our work stored and accessible.*

What is GitHub Codespaces?

GitHub Codespaces is a cloud-based development environment provided by GitHub. In simple terms, it gives you a fully set up coding workspace that runs right in your browser. You do not need to install anything on your computer. No local setup, no dependency issues. You just open a Codespace and start writing code.

When you create a Codespace from a GitHub repository, it spins up a virtual machine in the cloud with everything you need. It comes with a code editor (basically VS Code in the browser), a terminal, and all the tools you would normally have to install yourself. It feels just like working on your own machine, except everything is running in the cloud.

This is especially useful when you want a consistent environment. Everyone on the team gets the same setup, regardless of whether they are on Windows, Mac, or Linux. No more "it works on my machine" problems.

Key Features of GitHub Codespaces

This file is meant for personal use by [illegible]@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

- **Cloud-Based Environment:** Codespaces runs entirely in the cloud. You do not need to install any IDE, language runtime, or package on your local machine. Just open a browser and you are good to go.
- **Built-in VS Code Editor:** The editor inside a Codespace is a full version of VS Code. It supports extensions, themes, keyboard shortcuts, and everything else you would expect from VS Code on your desktop.
- **Customizable Configuration:** You can use a devcontainer.json file to define exactly what tools, extensions, and settings your Codespace should have. This means every person who opens the project gets the same environment automatically.
- **Integrated Terminal:** Each Codespace comes with a built-in terminal where you can run commands, install packages, start servers, and do everything you would normally do in a local terminal.
- **Port Forwarding:** Codespaces can forward ports from the virtual machine to your browser. So if you start a web server on port 8501 inside the Codespace, you can access it directly from your browser.
- **Persistent Storage:** Your work is automatically saved inside the Codespace. Even if you close the browser, you can come back later and pick up right where you left off.
- **Flexible Machine Types:** You can choose from different machine types depending on how much computing power you need. For most course work, the default configuration works just fine.

How We Use Codespaces in This Course

In this course, GitHub Codespaces is where we do most of our actual work. We use it to deploy our Streamlit app, which serves as the main interface for our project. On top of that, we also use Codespaces to run both the backend and the frontend of our application.

The nice thing about this approach is that you do not have to worry about setting up Python environments, installing packages, or configuring servers on your local machine. Everything runs inside the Codespace. You just open it up, run a few commands, and your app is live.

So to sum it up: GitHub holds our files, and GitHub Codespaces is where we actually build, run, and deploy everything. That is the setup for this course, and it keeps things simple and consistent for everyone.